

Crash Pack: Forward Deployed Engineer / AI Consultant

This is the condensed field manual for the forward deployed role. It covers the 20 percent of the track that answers 80 percent of the questions from real engagements. It opens with a groundwork chapter. That chapter defines the vocabulary for the rest of the pack. The pack then turns the seven chapters of Role Track 7 into decision rules, numbers, traps, drills, and one cheat sheet.

How to use it. Read sections 0, 1, 2, and 3 in order. Start with the words, then the engagement rhythm, then the rules, then the numbers. The numbers let you compute each decision.

Drill section 5 until every answer is instant. Print section 6 and keep it next to you in customer meetings. The full track holds the reference for every compressed rule here: scoping memos, worked Python, and labs.

Key takeaways

- Customers measure a forward deployed engineer by outcomes, not by model quality. Adoption, reliability in production, and time to value form the scorecard.
- Every decision on an engagement reduces to five rules. Scope with kill criteria. Let data classification pick the pattern. Price the work by the cost of each successful outcome. Gate releases on the acceptance eval. Pause features when an error budget burns.
- The customer decides what "works" means. Your eval must convince a skeptical VP, not a reviewer.

0. Groundwork: the words this pack leans on

This pack covers customer work. It uses a few words from the base volumes. Each word keeps its plain meaning everywhere after this.

Term	What it means
Eval	An eval is a structured measurement of what the system does. It runs on real cases and uses a grader.
Golden set	A golden set is a versioned collection of real customer cases with human-verified answers. Every eval rests on this contract.
Grader	A grader decides pass or fail on a golden-set case. Use deterministic code first. Use a calibrated model judge only if code cannot decide.
RAG	RAG means retrieval-augmented generation. The model answers from documents it retrieves at query time. It does not answer from memory.
Retrieval	Retrieval is the step that finds the relevant documents or passages for a query.
Agent	An agent is software that pursues a goal in a loop. It thinks, acts on tools, observes, then repeats.
Token	A token is the unit for counting text. Models bill and budget in tokens.
Context window	A context window is the working memory of the model for one call, measured in tokens.
Model call	A model call is one request to the language model.
Pilot	A pilot is a small deployment to real users. It runs to learn, before any go-live decision.
Shadow	A shadow is a deployment that observes real traffic and drafts answers, but serves nothing. Teams measure success rates from the pilot shadow.
Canary	A canary sends a small share of live traffic (often 5%) to the new version first. It rolls back automatically if the version misbehaves.
Rollback	A rollback restores the previous version. Target: restore in under 15 minutes. Test the restore before the canary starts.
Drift	Drift is the slow change of data, users, or model behavior after deployment. Drift in data quietly kills the quality SLOs.

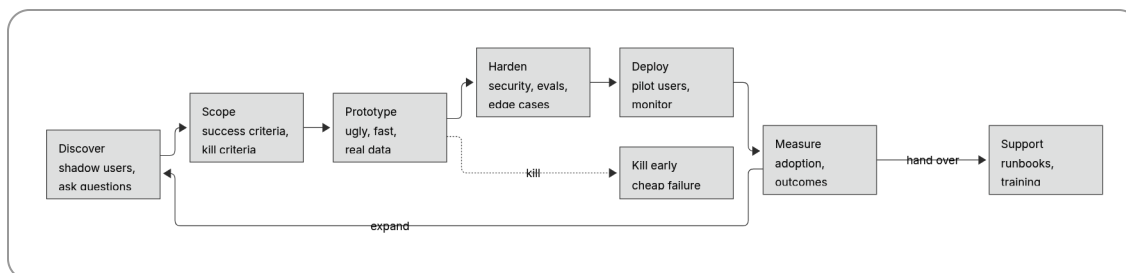
ROI	ROI means return on investment. It is what the customer gains, divided by what it costs.
SSO	SSO means single sign-on: one login that carries a user's identity across systems. Permissions usually connect through it.
VPC	VPC means virtual private cloud. It is the customer's own isolated section of a cloud provider, in the customer's account and region.
Throughput	Throughput is the count of requests that the system serves each second. Latency and throughput show the same system from different ends.
CRM	CRM means customer relationship management. The CRM system holds the customer's data. More copies exist than anyone admits.

1. The engagement loop

\n

Chapter 0 gave you the vocabulary of customer work. This chapter gives you the rhythm: every engagement follows the same loop of phases, from first contact to handover. Learn it once and you can run any of them.

Every engagement follows the same rhythm. Learn it once and you can run any of them.



How to read this diagram

1. **Discover** means watching real users do the job, not reading a requirements document. Nobody writes down the best requirements.
2. **Scope** turns discovery into a signed plan with numbers. The plan holds success criteria and kill criteria. The kill criteria are the most important line in the plan.
3. **Prototype** is intentionally ugly and fast, and it runs on the customer's real data. If it fails here, follow the dotted arrow: kill it cheaply. That is a good outcome.
4. **Harden** is where the demo becomes a system. Most of the calendar goes here. This step covers the security review, evals, and edge cases. It also covers monitoring and fallbacks.
5. **Deploy** starts with a small pilot, not a launch. **Measure** checks the success criteria from the scope step. Then expand or hand over with runbooks and training.

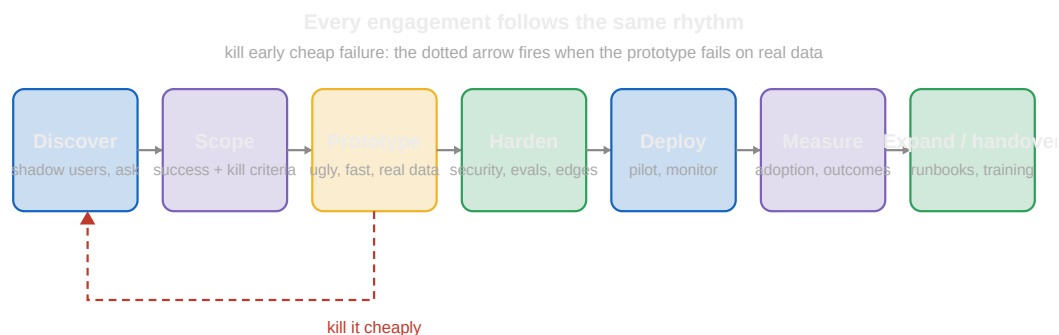


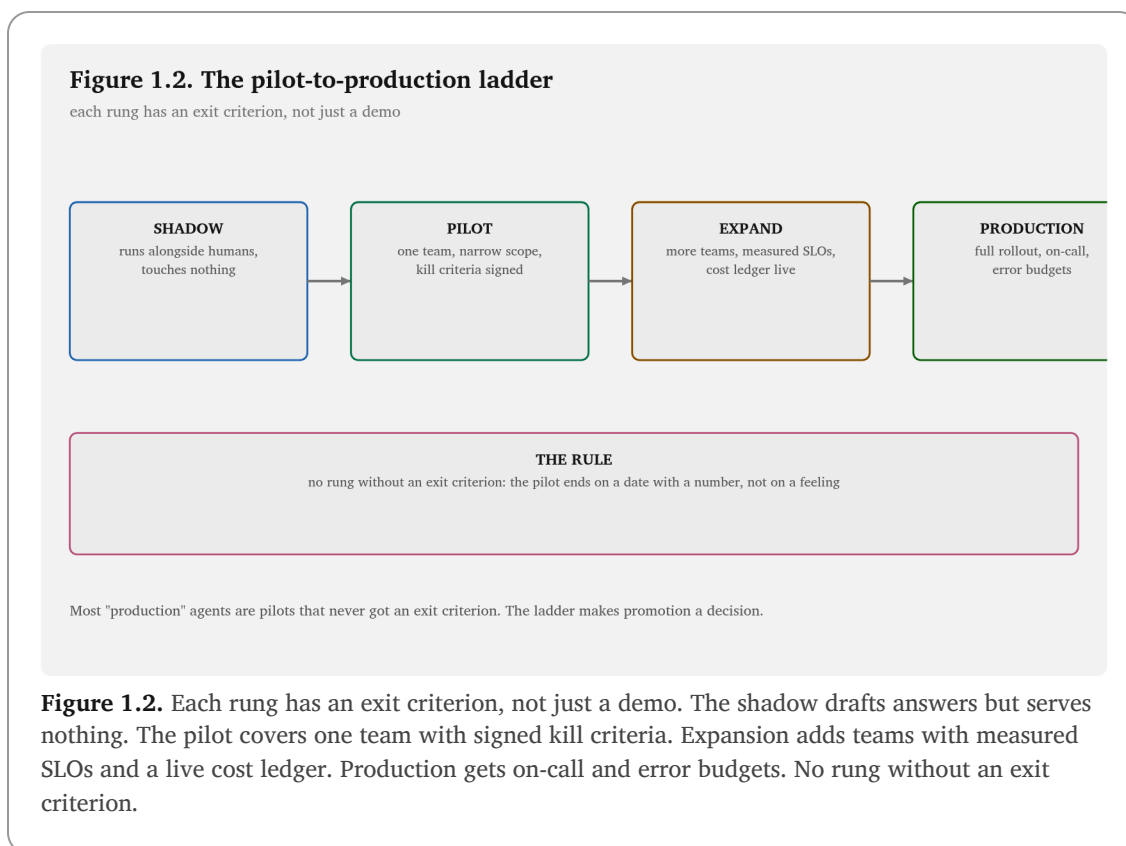
Figure 1.1. The engagement loop. Kill a bad engagement in week 2 and it costs a week. Kill it in month 4 and it costs the relationship. The kill criteria from scoping decide when the dashed arrow moves.

The scorecard for the whole loop, in five measures:

Measure	What to track	Why it decides the engagement
Customer outcomes	1-3 outcome metrics from scoping, reported weekly	The only score the sponsor cares about
Adoption	Weekly active users, share of eligible work, retention after month two	A system nobody uses failed, however excellent the tech
Reliability	Uptime, p95 latency (the latency 95% of requests beat), task success rate on real traffic, time to recover	The customer experiences your error rate, not your benchmark
Time to value	Days to first working prototype on real data (target: 14-21)	Engagements fail in long silent builds. Show early proof. It builds trust.
Expansion / handover	More teams buying in, or clean handover with runbooks	The two healthy endings. Everything else is a stall.

Next: Chapter 2 turns the loop into decisions. It covers scoping and deployment patterns. Next it turns to build vs buy. It then covers evals as contracts and escalation triggers.

\n



What to look at first: the four rungs left to right, then the red rule box. The one takeaway: most "production" agents are pilots that never got an exit criterion.

2. Decision rules

\n

The engagement loop tells you what happens when. This chapter tells you how to decide. Five decisions carry most engagements: the go / no-go rule, the deployment pattern, build vs buy, evals as contracts, and escalation triggers.

2.1 Scoping: the go / no-go rule

Discovery has three moves, in order:

\n

1. **Watch:** shadow 2-4 users. Watching first makes your questions specific.
2. **Ask:** run the 10-question bank.
3. **Map:** draw the workflow boxes, marked automate / assist / leave alone. Mapping last forces you to commit to what you learned.

\n

Never propose a technical approach before the map exists.

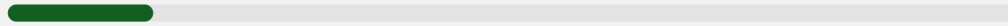
The go / no-go diamond sits at the end of the map:

Gate	Go	No-go (kill it, write why)
Automatable work	At least one box is pattern-matching, marked automate or assist	Every box needs judgment under uncertainty, or the signature is the point
Success criteria	Writable as numbers plus dates, measurable on customer data	Only vibes available ("improve efficiency")
Kill criteria	Specific conditions to stop, signed by the sponsor	Nobody will sign a stop condition
Data	Exists, located, access path approved or approvable	"In the CRM" means three CRMs and a spreadsheet with no owner (CRM: the customer relationship management system where their data lives)
Hard constraints	At least one deployment pattern survives the vetoes (2.2)	Residency, security, or procurement vetoes all four patterns
Economics	Cost per successful outcome beats the human baseline at real volume (3.1, 3.3)	Fixed costs dominate at their volume and always will
Sponsor	A named executive owns the outcome and the decision	No sponsor, or the sponsor delegates to a committee

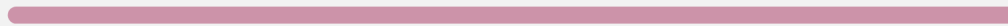
Kill-criteria rules: write kill criteria before building, give each one a number and a date, and get the sponsor to sign them. Example (on real data): "If the acceptance eval stays below 80 percent after 4 weeks, stop." Treat a triggered kill criterion as a success. Killing a bad engagement in week 2 costs a week. Killing it in month 4 costs the relationship. The most trusted engineers kill fast and say why.

Figure 2.8. Kill early or kill late

The kill criteria decide when. The calendar decides the price.

Kill in week 2

Cost: **one week**. The team moves on. The sponsor trusts you.

Kill in month 4

Cost: **the relationship**. Four months of budget burned on a zombie.

Write the kill criteria before building. Triggering one is a success, not a failure.

Figure 2.8. Kill a bad engagement in week 2 and it costs a week. Kill it in month 4 and it costs the relationship. The kill criteria decide when the arrow moves.

How to read this diagram

1. Top bar: the engagement dies in week 2. The green segment is short. The cost is one week.
2. Bottom bar: the same engagement dies in month 4. The red segment spans the whole timeline. The cost is the relationship.
3. What it is: a visual of the kill-criteria rule. Why it exists: teams delay the kill and pay later. What breaks: without written criteria, nobody pulls the trigger.

A scope without numbers is a wish. "The system improves support efficiency" cannot fail, which means it cannot succeed either. Every commitment you make in a demo becomes a requirement in production. Promise less than the demo shows. Write down exactly what the demo did not cover.

Scoping decides whether the engagement should exist at all. Once it exists, the next decision is where it runs: five constraints pick among four deployment patterns. Read the constraints before drawing any architecture.

\n

Figure 2.2. The go / no-go checklist

seven boxes ticked, or it is a no-go

☒	AUTOMATABLE WORK the task is worth automating and can be automated
☒	NUMERIC + DATED SUCCESS CRITERIA a number, a date, and who measures it
☒	SIGNED KILL CRITERIA what failure looks like, agreed before the pilot starts
☒	DATA LOCATED the data exists, is reachable, and is legal to use
☒	HARD CONSTRAINTS NAMED security, residency, identity: section 2.2, in writing
☒	ECONOMICS BEAT THE HUMAN BASELINE cost per successful outcome beats the human, at real volume
☒	SPONSOR NAMED a named executive owns the outcome and the decision

The kill criteria are the kindest part: they let a failing pilot die on schedule instead of on politics.

Figure 2.2. Automatable work. Numeric and dated success criteria. Signed kill criteria. Data located and legal to use. Hard constraints named in writing. Economics that beat the human baseline at real volume. A named sponsor. The kill criteria are the kindest part: they let a failing pilot die on schedule instead of on politics.

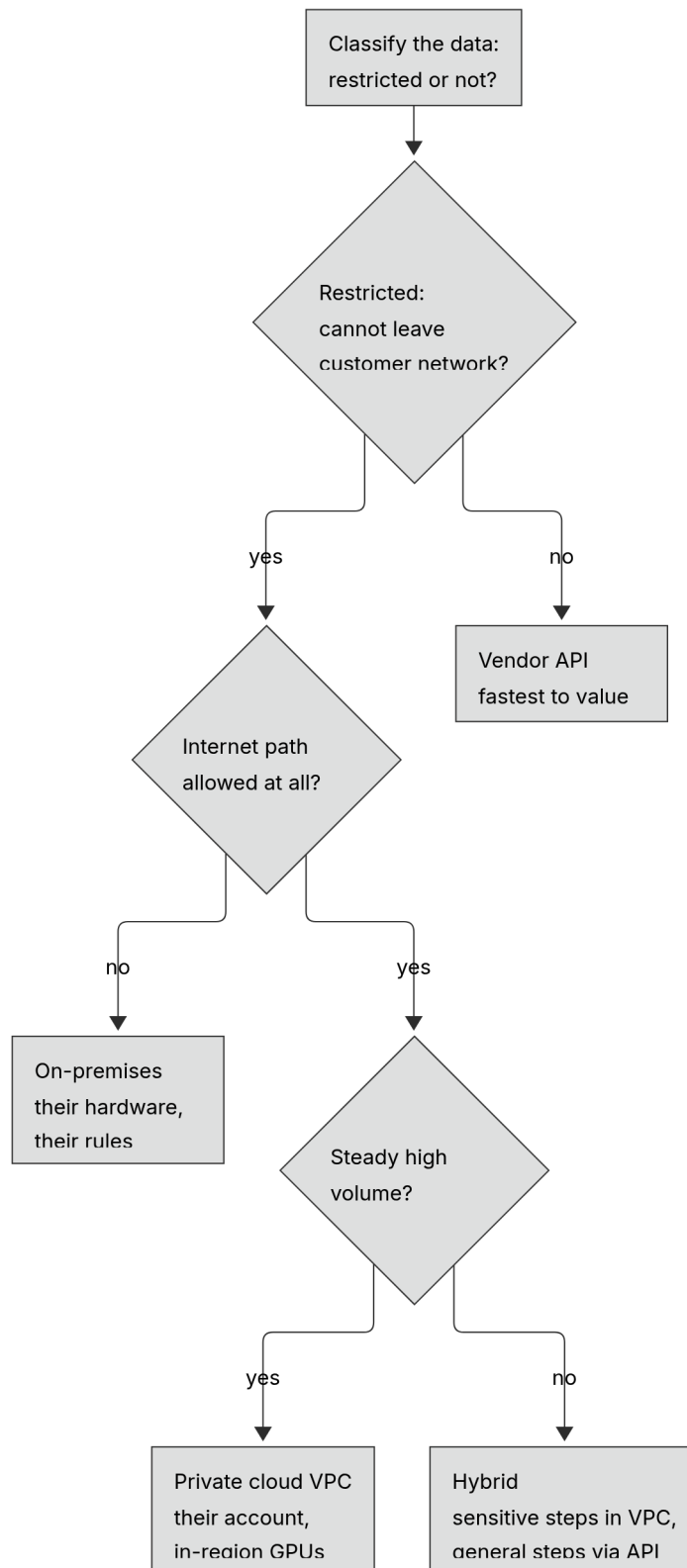
What to look at first: the seven checklist rows. The one takeaway: scope the exit before the entrance.

2.2 Deployment pattern: five constraints, four patterns

Read the customer's constraints before drawing any architecture. The big five, and the question each answers:

#	Constraint	The question to ask in week one	What it decides
1	Security review	What data flows where, who can read the logs?	Takes 2-6 weeks; start it in week one, not week eight
2	Data residency	Where is this data allowed to live?	Can kill an entire architecture; decides where the model runs
3	Legacy integration	What systems hold the data, and what interfaces exist?	Budget real time; it is always more than you think
4	Identity and access	What permissions does each user already have?	Permission check goes before retrieval, never after
5	Procurement	Are we inside an existing contract or starting a new one?	A pilot needing a purchase order in week two will stall

Four patterns cover nearly every engagement. Data classification picks the pattern:



How to read this diagram

1. The first question is always data classification. Everything else follows from it.
2. "Restricted" means regulated or contractually confined: health records, financial data, defense work, data under a strict data-processing agreement.
3. No internet path means on-premises. Accept it early; fighting it wastes months.
4. Volume decides between VPC and hybrid: steady high volume justifies dedicated GPUs in the VPC (the customer's own isolated cloud section, billed to their account); spiky or mixed workloads fit a hybrid split.
5. Unrestricted data goes to the vendor API. Do not build infrastructure for a problem the API already solves.

The selection rule is **veto, then score**. Hard constraints are vetoes, not weights: "data cannot leave the network" eliminates the vendor API entirely, no matter how fast it is. Score only what survives, with weights you set live with the customer and can defend out loud. A scored table the customer can argue with beats a recommendation they cannot inspect.

For the security review meeting, bring three artifacts: a data-flow diagram (every arrow labeled with what flows, encrypted in transit and at rest), a data inventory (what you store, where, for how long, including prompts and eval traces), and the permission model ("we check the same access list the source system uses, before retrieval" ends meetings early, in a good way).

Figure 2.3. The five enterprise constraints

name them in week one or meet them in month six



Four deployment patterns exist because these five constraints combine differently. The pattern follows the constraints, never the reverse.

Figure 2.3. Name them in week one or meet them in month six. The four deployment patterns exist because these five constraints combine differently; the pattern follows the constraints, never the reverse.

What to look at first: the five rows. The one takeaway: every one of these has killed a deployment that ignored it.

2.3 Build vs buy

Model "buy" as one more option in the cost comparison. Count zero build cost. Use the vendor's per-task price as the model cost. Discount the vendor's claimed success rate with your skepticism, then validate it in the pilot shadow. The honest answer is usually hybrid: **buy the commodity model, build the evals, the integration, and the permission layer yourself.** That is where a deployed engineer adds the most value.

Build when	Buy when
The workflow is the differentiator (their process, their data, their permissions)	The capability is a commodity (a model API, a vector database)
Evals must match the customer's definition of done (they always must)	The vendor's success rate survives your pilot-shadow measurement
Data cannot leave their network and no compliant vendor option exists	Volume is low and fixed build cost would dominate (check 3.3)
The integration surface is the product (legacy systems, SSO (single sign-on), audit)	Time to value matters more than differentiation for this step

Build vs buy settles who builds the system. The next question is what "done" means. This section turns acceptance thresholds into contracts the customer signs before building starts.

\n

Figure 2.4. Build vs buy: the decision matrix

buy the commodity, build the moat

BUILD	BUY
build when it is your differentiator	buy when it is undifferentiated heavy lifting
build when the vendor cannot meet a hard constraint	buy when the vendor meets all five constraints
build when you will iterate on it weekly	buy when it changes slowly and boringly
THE TIEBREAK total cost of ownership over 3 years, including the team that maintains it	

Figure 2.4. Build when it is your differentiator, when the vendor cannot meet a hard constraint, or when you will iterate weekly. Buy undifferentiated heavy lifting that changes slowly. The tiebreak is total cost of ownership over three years, including the team that maintains it.

What to look at first: the build column versus the buy column, row by row. The one takeaway: buy the commodity, build the moat.

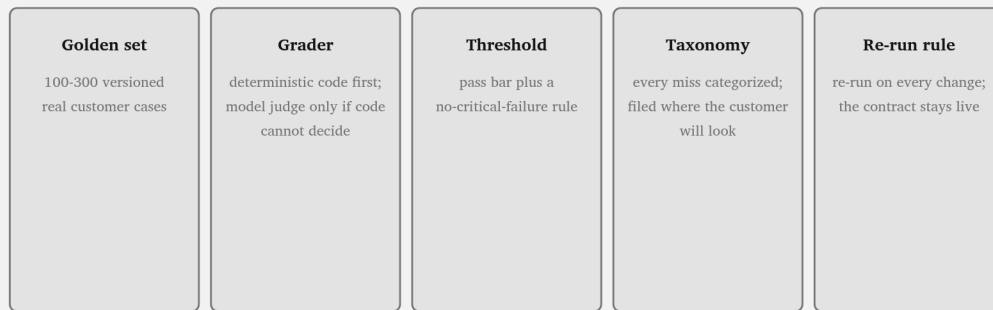
2.4 Evals as contracts: acceptance thresholds

The eval is a contract. Both sides sign it in advance. It states what gets measured, on what data, and with what pass threshold. It states what happens on pass or fail. Five parts, all agreed before building:

Part	Threshold / rule	Why it is load-bearing
Golden set	100-300 real customer cases, labeled by customer experts	Below 100 the confidence intervals are too wide to argue about; above 300 the experts never finish
Grader	Deterministic first (exact match, rubric code); model judge only if impossible, calibrated against humans first	A grader the customer cannot read is a grader they will not trust
Pass threshold	A number plus the no-critical-failure rule	One catastrophic miss fails the eval at any average; customers remember the catastrophe
Failure taxonomy	Every miss categorized: wrong answer, refused, wrong question, too slow, leaked data	"9 formatting edge cases, 3 reasoning gaps" is a fix list; "87 percent" is a shrug
Re-run rule	On every model or prompt change, on a schedule, before every pilot release	The eval is a gate in the pipeline, not a one-time ceremony

Figure 2.9. The eval-as-contract: five parts

All five are agreed before building. All five are co-signed.



An eval the customer did not sign is a wish.

Figure 2.9. The eval is a contract with five parts: the golden set, the grader, the threshold plus the no-critical-failure rule, the taxonomy, and the re-run rule. All agreed before building. All co-signed.

How to read this diagram

1. Box 1, golden set: 100-300 versioned real customer cases. The contract behind every eval.
2. Box 2, grader: deterministic code first. A calibrated model judge only if code cannot decide.
3. Box 3, threshold: the pass bar plus the no-critical-failure rule, checked as a separate boolean.
4. Box 4, taxonomy: every miss categorized, filed where the customer will look.
5. Box 5, re-run rule: re-run on every change. The contract stays live.
6. What it is: the anatomy of an eval both sides sign. Why it exists: unsigned evals are wishes. What breaks: a missing re-run rule lets the eval rot.

Run user acceptance in three rounds: **blind grading** (3-5 experts grade a shuffled mix of system and human outputs, killing bias both ways), **adversarial session** (invite the skeptics to break it for 30 minutes; every break becomes a golden case or a documented limit), **pilot shadow** (two weeks drafting alongside the current process, human decides; the agreement rate is the number the sponsor will quote).

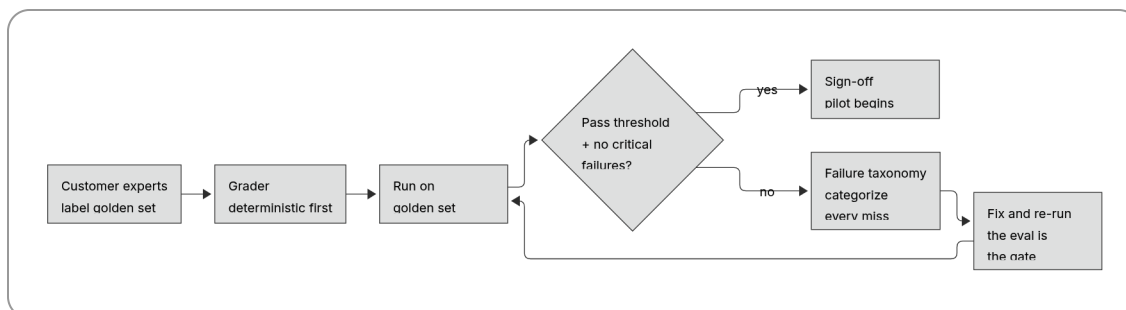


Figure 2.5. Evals as contracts: acceptance thresholds

the customer signs the definition of done

GOLDEN SET 100-300 fixed version, covers the customer's real tasks
DETERMINISTIC GRADER same input, same score; temperature 0, pinned version
PASS THRESHOLD + NO-CRITICAL-FAILURE a number AND a rule that one critical miss fails the batch
FAILURE TAXONOMY every miss filed where the customer will look for it
RE-RUN RULE schedule plus after every model or prompt change

An eval the customer did not sign is a wish. The contract turns "it works" into "it works by this measure, on these tasks."

Figure 2.5. A golden set of 100-300 versioned items on the customer's real tasks. A deterministic grader. A pass threshold plus a no-critical-failure rule. A failure taxonomy filed where the customer will look. A re-run rule. An eval the customer did not sign is a wish.

What to look at first: the five contract clauses. The one takeaway: the contract turns "it works" into "it works by this measure, on these tasks."

2.5 Escalation triggers

Escalation is a rule, not a feeling. Set the lines in the scoping memo, then let the dashboard fire them. Read the table as a ladder: each row is a worse failure state than the last, and each action costs more than the last.

Trigger	Line	Action
Error budget burn rate above 2x	Sustained, any of the three budgets	Pause feature work; fix reliability; tell the sponsor the same day
Quality below SLO (service-level objective: the reliability target you promise) on sampled traffic	Page, not ticket	Wake someone; the runbook (the step-by-step recovery playbook for on-call) must already exist
Kill criterion approaching	The week you see it, not the week it hits	Tell the sponsor with a new date and what changed, or start the kill
Acceptance eval fails the bar	Any release gate	Lead with the number, then the taxonomy, then the plan and the cost
Security review blocked	Week one, not week eight	Escalate for a parallel path (on-prem option, narrower data scope)
Scope creep ("can it also do invoices?")	Immediately	Point at the scoping memo; new workflow means a new memo and new criteria

The incident ritual stays the same every time:

1. **Mitigate first:** roll back, pause, or route to human handoff.
2. **Write the timeline** with facts only.
3. **Find the cause** with five whys, blameless.
4. **Add a fuzzer probe** and usually a golden-set case at the exact break. A fuzzer probe is an automated adversarial-input test that hunts for crashes and hangs.
5. **Share the post-mortem** with the customer within 48 hours.

Customers forgive incidents; they do not forgive silence.

Figure 2.10. The incident ritual: same every time

Five steps, in order. Customers forgive incidents, not silence.



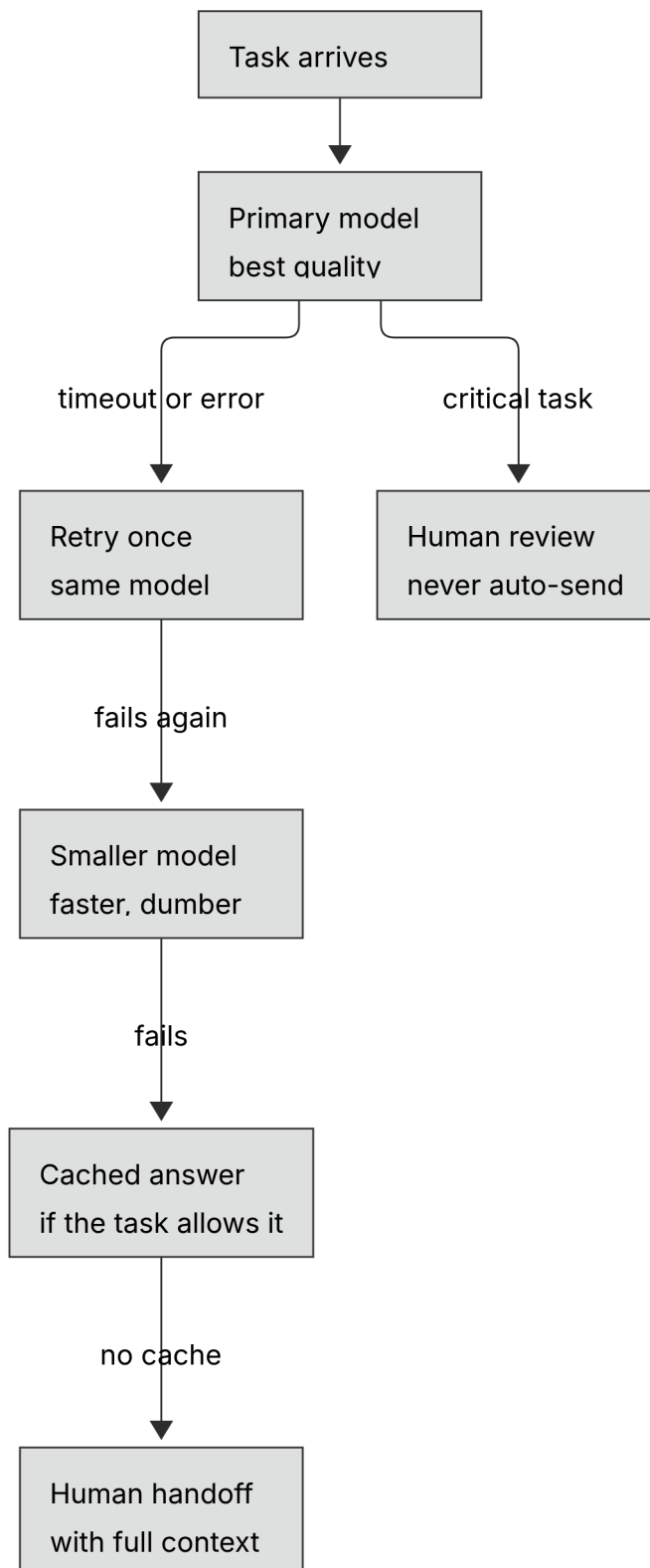
A fuzzer probe is an automated adversarial-input test that hunts for crashes and hangs.

Figure 2.10. The incident ritual, same every time: mitigate, timeline, cause, probe, share. Customers forgive incidents. They do not forgive silence.

How to read this diagram

1. Step 1, mitigate: roll back, pause, or route to human handoff. Stop the bleeding first.
2. Step 2, timeline: write what happened with facts only. No blame yet.
3. Step 3, cause: run five whys, blameless. Find the mechanism, not the person.
4. Step 4, probe: add a fuzzer probe and usually a golden-set case at the exact break.
5. Step 5, share: send the post-mortem to the customer within 48 hours.
6. What it is: the ordered response to any incident. Why it exists: panic skips steps. What breaks: skipping the probe lets the same incident return.

The fallback cascade is the ordered list of what happens when an AI call fails. Design it before the pilot.



How to read this diagram

1. Order the cascade by cost and reliability: each step costs less and works more reliably than the last, usually at lower quality. Agree the trade-off in advance.
2. **Retry once**, never in a loop. Retry storms are how one slow model takes down the whole system.
3. The **smaller model** step exists because a fast, dumber answer beats no answer for most tasks. Know which tasks tolerate it (drafting) and which do not (sending to a customer).
4. The **human handoff** is the bottom of the cascade, not a failure. It carries full context: what the team tried, what failed, the partial work so far.
5. **Critical tasks** skip sideways into mandatory human review. Decide the critical list in scoping, not at 2am.

Ordered by cost and reliability: each step is cheaper and more reliable than the last

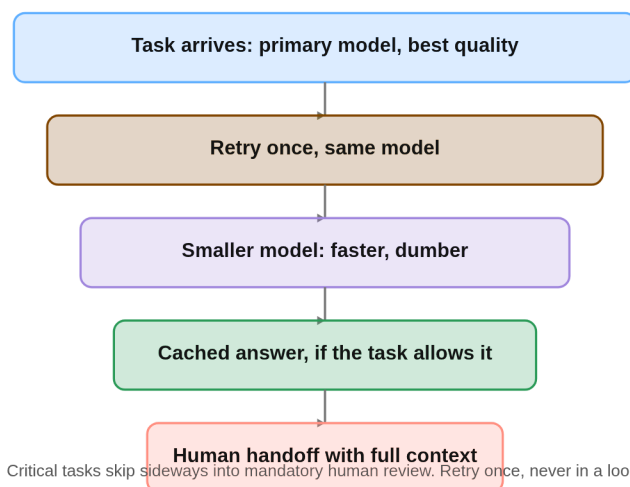


Figure 2.6. The fallback cascade, ordered by cost and reliability. Retry once, never in a loop: retry storms are how one slow model takes down the whole system. Critical tasks skip sideways into mandatory human review.

Next: Chapter 3 puts numbers behind the decisions: cost per outcome, error budgets, breakeven math, and serving costs.

\n

Figure 2.6. Escalation triggers: when the agent hands back to a human

silence is not a strategy



Escalation is a feature, not an apology. Design the handoff before the failure, and the on-call thanks you.

Figure 2.7. If confidence falls below the bar, hand off. If the situation is novel, hand off. If the action is high-stakes, hand off. If the failure repeats, hand off. Each handoff carries state, options tried, and a recommendation, never a bare error. Escalation is a feature, not an apology. Design the handoff before the failure, and the on-call thanks you.

What to look at first: the four trigger boxes, then the green handoff box. The one takeaway: silence is not a strategy.

3. Must-memorize numbers and formulas

\n

Decisions need numbers behind them. This chapter is the arithmetic of the FDE role. It covers what each successful outcome costs, what failures may cost, and when the economics beat the human baseline.

3.1 Cost per successful outcome

The unit that matters is cost per successful outcome: the total cost of one task completed correctly, in dollars per successful task. Not cost per token. Not cost per API call. Five parts feed it:

cost per successful outcome

model: tokens in + out	\$0.12
infra share: monthly / tasks	\$0.20
build share: amortized	\$0.25
maintenance share	\$0.15
review labor: 92% x 1.5 min	\$1.73
rework labor: 8% x 15 min	\$1.50
per attempt: \$3.95	
/ success rate 0.92	
= \$4.29 per successful task	

The formula, per task attempt:

$$\text{per_attempt} = \text{model} + \text{infra_share} + \text{build_share} + \text{maint_share} + \text{success_rate} \times \text{review_cost} + (1 - \text{success_rate}) \times \text{rework_cost}$$

$$\text{cost_per_outcome} = \text{per_attempt} / \text{success_rate}$$

\n

where:

- **model**: the model cost per attempt in dollars.
- **infra_share**: monthly infra divided by tasks.
- **build_share**: the amortized build cost.
- **maint_share**: maintenance.
- **success_rate**: the acceptance-eval success fraction.
- **review_cost**: the labor cost of reviewing one success.
- **rework_cost**: the labor cost of reworking one failure.

The ASCII box above works it: \$3.95 per attempt / 0.92 = \$4.29 per successful task.

Figure 3.6. Cost per successful outcome: the worked example

Divide by the success rate, not by the attempt count.

per attempt

\$3.95

$\xrightarrow{/ 0.92}$ model + infra + build + maint
+ review + rework shares

per successful outcome

\$4.29

\$3.95 per attempt / 0.92 success rate
= **\$4.29 per successful task**

Success rates come from your acceptance eval. Vendor benchmarks make downstream numbers fiction.

Figure 3.6. Cost per attempt divided by the success rate gives cost per successful outcome. Quote per successful outcome in every business case.

Dividing by the success rate is the step people skip. It converts "cost per attempt" into "cost per outcome," which is what the customer's budget experiences. Success rates must come from your acceptance eval on customer data; vendor benchmarks make every downstream number fiction. Human review minutes are the most uncertain input: measure them in the pilot shadow, do not estimate them in a meeting (people are optimistic about review speed by 2-3x).

\n

Interview line to memorize. Success rates must come from your acceptance eval on customer data. Vendor benchmarks make every downstream number fiction.

```

# cost_per_outcome.py -- the number the budget feels (executed, output below)
#
# WHAT: Cost per successful outcome for one design option.
# WHY: Token-price comparisons mislead. A cheap model at 70% success with
#       heavy human review usually loses to a pricier model at 92% that
#       just works. Every assumption is a named parameter the customer can
#       challenge in the meeting.
from dataclasses import dataclass

@dataclass
class Option:
    name: str
    model_cost: float      # $ model cost per task attempt (measured tokens x
price)
    success_rate: float    # from the ACCEPTANCE EVAL, not vendor benchmarks
    review_min: float      # human minutes to review one successful task
    rework_min: float      # human minutes to rework one failed task
    labor_hr: float = 75.0 # fully-loaded reviewer $/hour (illustrative)

def cost_per_outcome(o, infra_share, build_share, maint_share):
    # Labor splits on the outcome: successes need review, failures need rework.
    labor = (o.success_rate * o.review_min / 60 * o.labor_hr
            + (1 - o.success_rate) * o.rework_min / 60 * o.labor_hr)
    per_attempt = o.model_cost + infra_share + build_share + maint_share +
labor
    # A zero success rate means infinite cost per outcome. The assert keeps
    # the math honest instead of smoothing a dead option into the ranking.
    assert o.success_rate > 0, "zero success rate = infinite cost per outcome"
    return per_attempt / o.success_rate # attempts -> outcomes: the key
division

# Program: 20k tasks/mo, $4k infra, $120k build over 24 months, $3k/mo upkeep
tasks, infra_m, build, life_m, maint_m = 20_000, 4_000.0, 120_000.0, 24,
3_000.0
shares = (infra_m / tasks, build / life_m / tasks, maint_m / tasks)
cheap = Option("small model + heavy review", 0.02, 0.70, 4.0, 15.0)
pricey = Option("large model + light review", 0.12, 0.92, 1.5, 15.0)
for o in (cheap, pricey):
    print(f"{o.name}: ${cost_per_outcome(o, *shares):.2f}/successful task")

```

Output (verified by execution):

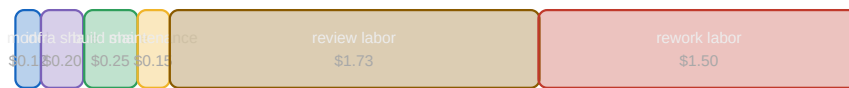
```

small model + heavy review: $13.92/successful task
large model + light review: $4.29/successful task

```

The 6x token-price gap reverses: the pricey model wins because its higher success rate slashes rework labor. That is the token-price fallacy in one table.

$\$3.95 \text{ per attempt} / 0.92 \text{ success} = \$4.29 \text{ per successful task}$



per attempt \$3.95

dividing by the success rate is the step people skip

cost per successful outcome = per-attempt cost / success rate

Human review minutes are the most uncertain input: measure them in the pilot shadow.

Figure 3.1. The five parts of cost per successful outcome. Human labor (review plus rework) dwarfs the model cost, which is why the pricey model at 92% success beats the cheap model at 70%.

Cost per outcome tells you what each success costs. The next number tells you what failures may cost: error budgets, and the burn rate that guards them.

\n

Figure 3.2. Cost per successful outcome: $\$3.95 / 0.92 = \4.29

divide by the success rate, not by the attempt count

cost per attempt: \$3.95

success rate: 92%

cost per SUCCESS: $\$3.95 / 0.92 = \4.29

The attempt cost flatters; the success cost tells the truth. A cheap attempt with a 50% success rate costs double per outcome.

Quote per successful outcome in every business case. The CFO will ask; have the number ready.

Figure 3.2. Divide by the success rate, not by the attempt count. The attempt cost flatters; a cheap attempt with a 50% success rate costs double per outcome. Quote per successful outcome in every business case: the CFO will ask, so have the number ready.

What to look at first: the gray attempt bar versus the blue success bar, then the amber result. The one takeaway: the success cost tells the truth.

3.2 Error budgets and burn rates

AI SLOs need three dimensions, because the system has three failure states (down, slow, wrong-looking answer):

Dimension	Example SLO (30 days)	Budget (1 - SLO)	What burns it
Availability	99.9% of requests answered	0.1% ~ 43.2 min	Outages, deploy failures, quota exhaustion
Latency	95% of tasks under 30 s	5% slow tasks	Long inputs, model slowness, retry storms
Quality	90% of sampled production tasks pass the grader	10% bad answers	Prompt drift, data drift, model version change

Budget math: 99.9% availability over 30 days allows $0.001 \times 30 \times 24 \times 60 = 43.2$ minutes of downtime. Memorize that number.

Burn rate = (observed failure fraction) / (allowed failure fraction). Units: a dimensionless multiple. 1.0 means exactly on pace to exhaust the budget at the end of the window.

Above 1.0 means breaching early.

burn_rate = (observed failure fraction) / (allowed failure fraction)

where: **burn_rate** is a dimensionless multiple; **observed failure fraction** is what you measure; **allowed failure fraction** is what the budget permits. 1.0 means exactly on pace to exhaust the budget at the end of the window.

Worked example. A 99.9% availability SLO allows 0.1% failures. If you observe 0.3% failures, $\text{burn_rate} = 0.3 / 0.1 = 3x$. The budget burns three times too fast. The agreed line triggers: burn above 2x pauses feature work. Alert on burn rate, never on raw counts.

Alert on burn rate, not raw counts. A quality budget that burns at 2x with three weeks left in the month will breach before the month ends. The agreed line: **burn above 2x pauses feature work** until it drops under 1x. That rule lives in the scoping memo. The script just reads the dashboard.

\n

The burn-rate rule. Burn above 2x pauses feature work until it drops under 1x. The rule lives in the scoping memo, signed by the sponsor; the script just reads the dashboard.

```
# burn_rate.py -- error-budget burn rates (executed, output below)
#
# WHAT: Burn rate for one budget from observed failures.
# WHY: "The system feels flaky" loses arguments. "Quality is burning at
#       2.5x; per our agreement, features pause until it is under 1x" wins.
def burn_rate(observed_fail, total, slo):
    # observed_fail/total: measured failure fraction in the window.
    # 1 - slo: the allowed fraction. Their ratio is the burn multiple.
    allowed = 1.0 - slo
    if allowed == 0: # a 100% SLO: any failure is a breach, report infinity
        return float("inf") if observed_fail else 0.0
    return (observed_fail / total) / allowed

# Availability: SLO 99.9%, 5 unanswered of 1000 requests in the window
print("availability burn:", round(burn_rate(5, 1000, 0.999), 2), "x")
# Quality: SLO 90% pass, random sample of 100 production tasks, 25 failed
qb = burn_rate(25, 100, 0.90)
print("quality burn:      ", round(qb, 2), "x",
      "-> PAUSE FEATURES" if qb > 2.0 else "")
```

Output (verified by execution):

```
availability burn: 5.0 x
quality burn:      2.5 x -> PAUSE FEATURES
```

Note the quality sample: you cannot grade every production task, so you grade a random sample and always report n. Grading only the easy traffic makes the quality SLO a lie the dashboard tells.

Figure 3.3. Burn rate: observed errors over allowed errors

99.9% over 30 days is 43.2 minutes of budget



$$\text{burn rate} = 26 / 43.2 = 0.60$$

below 1.0: on track | above 1.0: the budget dies early

THE POLICY

burn rate above 1.0 freezes non-urgent deploys; above 2.0 pages the on-call

Error budgets turn reliability arguments into arithmetic. The budget is spent by deploys, so gate the deploys on the burn rate.

Figure 3.3. An SLO of 99.9% over 30 days allows 43.2 minutes of errors. Compute burn rate as observed over allowed: below 1.0 the budget survives, above 1.0 it dies early. Above 1.0 freezes non-urgent deploys; above 2.0 pages the on-call.

Figure 3.7. Burn-rate zones: what each zone triggers

Burn rate = observed failures over allowed failures.



Example: 0.3% observed over 0.1% allowed = 3x. The agreed line triggers.

Alert on burn rate, not raw counts.

Figure 3.7. Burn below 1.0: the budget survives. Burn between 1.0 and 2.0: freeze non-urgent deploys. Burn above 2.0: page the on-call. Alert on burn rate, not raw counts.

How to read this diagram

1. Zone 1, below 1.0: the budget survives. Keep shipping.
2. Zone 2, 1.0 to 2.0: freeze non-urgent deploys. The budget dies early at this pace.
3. Zone 3, above 2.0: page the on-call. The budget breaches far too fast.
4. What it is: the policy ladder for error budgets. Why it exists: raw counts do not show pace. What breaks: alerting on breach instead of burn rate arrives too late.

What to look at first: the amber burned segment against the full budget bar, then the burn-rate number. The one takeaway: error budgets turn reliability arguments into arithmetic.

3.3 Breakeven vs the human baseline

"Is it worth it for our small team?" Solve for the monthly volume where the AI option becomes cheaper than the human:

$$n = \text{fixed_monthly} / (\text{human_per_success} - \text{var_per_success})$$

where $\text{fixed_monthly} = \text{infra} + \text{maintenance} + \text{build}/\text{lifetime_months}$, and $\text{var} = \text{model} + \text{success_rate} \times \text{review_cost} + (1 - \text{success_rate}) \times \text{rework_cost}$, all divided by success_rate .

Units: tasks per month. Below n , fixed costs dominate and the human wins. If the denominator is zero or negative, the AI never wins on cost: say so early.

```
# breakeven.py -- the honest answer to "is it worth it?" (executed, output
below)
#
# WHAT: Monthly task volume where the AI option beats the human baseline.
# WHY: Fixed costs (infra, build, upkeep) dominate at low volume. This is
#       the number that stops a bad engagement before it starts.
def breakeven(model_cost, success_rate, review_min, rework_min, labor_hr,
              fixed_monthly, human_per_task):
    # Variable cost per attempt, converted to per SUCCESSFUL task.
    var = (model_cost + success_rate * review_min / 60 * labor_hr
           + (1 - success_rate) * rework_min / 60 * labor_hr)
    denom = human_per_task - var / success_rate
    # denom <= 0: variable cost alone exceeds the human. No volume fixes that.
    return None if denom <= 0 else fixed_monthly / denom

fixed = 4_000.0 + 3_000.0 + 120_000.0 / 24 # infra + upkeep + build/24mo
n = breakeven(0.12, 0.92, 1.5, 15.0, 75.0, fixed, 12.50)
print("large model option:", f"{int(n):,} tasks/month" if n else "never
cheaper")
n2 = breakeven(0.02, 0.70, 4.0, 15.0, 75.0, fixed, 12.50)
print("small model option:", f"{int(n2):,} tasks/month" if n2 else "never
cheaper")
```

Output (verified by execution):

```
large model option: 1,353 tasks/month
small model option: never cheaper
```

The cheap model never beats the human: its variable cost per successful task already exceeds the human's \$12.50. No volume rescues it.

Breakeven tells you when the economics beat the human baseline. The next question is whether the system is usable at all: latency budgets per step and the serving cost model that prices them.

\n

Figure 3.4. Breakeven: $n = \text{fixed} / (\text{human} - \text{variable})$

how many tasks before the build pays for itself

$$n = \text{fixed cost} / (\text{human cost per task} - \text{variable cost per task})$$

fixed cost

the build: eng time, infra, evals

human cost / task

the fully-loaded human baseline

variable cost / task

inference + review per task

If the denominator is small, the breakeven is far: cheap human tasks are the hardest to justify automating.

Recompute breakeven quarterly: model prices fall, which moves the line in your favor.

Figure 3.4. How many tasks before the build pays for itself. If the denominator is small, the breakeven is far: cheap human tasks are the hardest to justify automating. Recompute quarterly, because model prices fall and move the line in your favor.

What to look at first: the formula, then the three named terms. The one takeaway: the denominator decides whether the project exists.

3.4 Latency budgets and the serving cost model

SLO latency on **p95 and p99, never the mean**. AI latency has a long tail: p50 looks fine while p95 is terrible because long inputs generate long outputs. The mean hides the exact requests the customer's executives tried in the demo. Always clarify whether the SLO names time-to-first-token or full generation. At 256 output tokens, decode physics sets a floor near **5.4 seconds** for full generation. If the customer wants sub-second, answer with streaming tokens, shorter outputs, or a smaller model, not hope.

The one-minute serving cost model (memorize it):

$$\text{cost per 1k requests} = (\text{GPUs} \times \$/\text{GPU-hr}) / (\text{req/s} \times 3.6)$$

Example: $9 \text{ GPUs} \times \$2.50/\text{hr} / (40 \text{ req/s} \times 3.6) = \$22.50 / 144 \approx \mathbf{\$0.16 \text{ per 1k requests}}$

Cost levers in ROI order:

1. **Batching** (group requests to fill a GPU): the biggest lever until the latency knee. The latency knee is the batch size where latency starts climbing faster than throughput.
2. **Output length**: linear in cost.

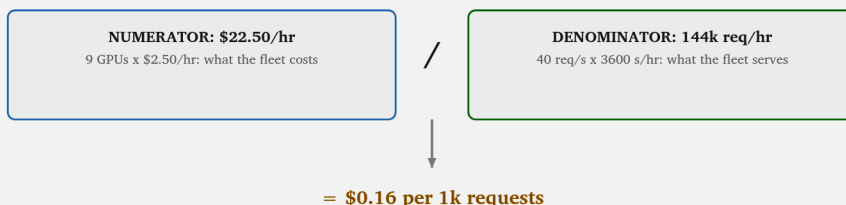
3. **Quantization** (serve the model in fewer bits per weight): roughly 2x per step down the ladder, with quality risk.
4. **Right-size context**: retrieved passages and agent histories are KV-cache memory. The KV cache holds the stored attention keys and values. Long contexts eat GPU memory before generation even starts.
5. **Caching**: exact-match and semantic. Semantic caching matches by meaning, so near-duplicate queries share one cached answer.
6. **Model routing**: use the small model for easy queries.

Latency and throughput (requests served per second) are the same problem viewed from different ends. Every throughput lever is a latency lever. Batch size is the dial that trades them.

Figure 3.5. The serving cost model: $(9 \times \$2.50/\text{hr}) / (40 \text{ req/s} \times 3.6) = \0.16 per 1k

GPUs per hour over requests per hour

$$(9 \text{ GPUs} \times \$2.50/\text{hr}) / (40 \text{ req/s} \times 3600 \text{ s/hr}) = \$0.16 / 1\text{k requests}$$



Utilization is the whole game: the same 9 GPUs at 10 req/s cost 4x per request. Autoscale to the demand curve.

Figure 3.5. GPUs per hour over requests per hour: \$22.50 of fleet per hour divided by 144k requests per hour. Utilization is the whole game. The same 9 GPUs at 10 req/s cost four times as much per request. Autoscale to the demand curve.

What to look at first: the blue numerator box and the green denominator box feeding the amber result. The one takeaway: idle GPUs are the most expensive line item.

3.5 Drift and rollout numbers

$$\text{PSI} = \text{sum over bins of } (\text{actual}\% - \text{expected}\%) \times \ln(\text{actual}\% / \text{expected}\%)$$

where: **PSI** is the population stability index; **actual%** and **expected%** are the serving and training fractions in one bin; the sum runs over all bins. Rule of thumb: below 0.10 no significant change, 0.10-0.25 small change worth watching, above 0.25 significant shift, investigate.

Worked example. Training split 50/50, serving split 60/40. Bin 1: $(0.60 - 0.50) \times \ln(0.60/0.50) = 0.10 \times 0.1823 = 0.0182$. Bin 2: $(0.40 - 0.50) \times \ln(0.40/0.50) = -0.10 \times -0.2231 = 0.0223$. $\text{PSI} = 0.0182 + 0.0223 = \mathbf{0.04}$: below 0.10, no significant change. At a 70/30 serving split the same math gives $\text{PSI} \approx 0.17$: a small change, worth watching.

```
# psi.py -- drift check with the worked numbers (executed, output below)
#
# WHAT: Population Stability Index between training and serving bins.
# WHY: Data drift is the quiet killer of quality SLOs. PSI turns "the data
#       feels different" into a number with an investigate line at 0.25.
import math

def psi(expected, actual):
    # expected/actual: bin fractions summing to 1.0 each. Bins are fixed
    # from training; changing bins between runs makes PSI meaningless.
    return sum((a - e) * math.log(a / e) for a, e in zip(actual, expected))

print("PSI mild shift: ", round(psi([0.50, 0.30, 0.20], [0.40, 0.35, 0.25]),
3))
print("PSI severe shift:", round(psi([0.50, 0.30, 0.20], [0.20, 0.30, 0.50]),
3))
```

Output (verified by execution):

```
PSI mild shift: 0.041
PSI severe shift: 0.55
```

0.041 is no significant change. 0.55 is well past the 0.25 investigate line.

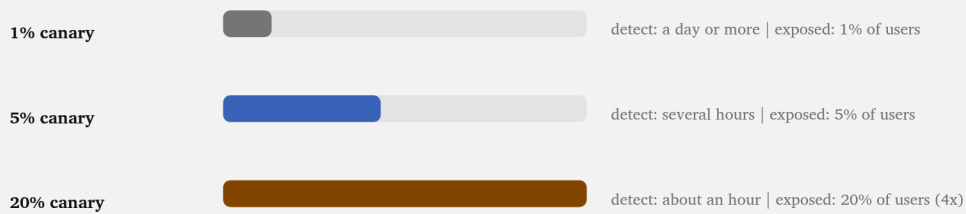
Canary math. A canary sends a small share of live traffic to the new version first. Baseline error rate 1%, and you want the canary to catch a regression to 2% before full rollout.

Telling 1% from 2% reliably needs roughly **2,000-5,000 samples** (binomial noise). At a 5% canary on 10,000 requests/hour, the canary sees 500 requests/hour, so detection takes several hours, not minutes.

Shrink the canary to 1% and detection takes a day or more. Grow it to 20% and you detect in about an hour, but you expose 4x the users. The trade is always detection speed vs blast radius. Rollout ladder: **shadow** (observes, serves nothing), then **canary** (small live fraction vs control), then **full** (staged 25/50/100%). Arm a tested one-command rollback before the canary starts. Rollback target: previous version restorable in **under 15 minutes**.

Figure 3.8. Canary size: detection speed vs blast radius

Small canary: slow to detect, few users exposed. Large canary: the reverse.



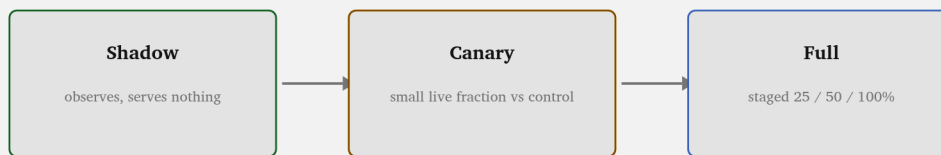
Telling 1% from 2% reliably needs roughly **2,000-5,000 samples** (binomial noise).

The trade is always detection speed vs blast radius.

Figure 3.8. Small canary: slow detection, few users exposed. Large canary: fast detection, many users exposed. The trade is always detection speed vs blast radius.

Figure 3.9. The rollout ladder

Each rung has an exit criterion. The rollback is armed before the canary.



Rollback target: previous version restorable in under 15 minutes.

Arm and test the rollback before the canary starts, not during the incident.

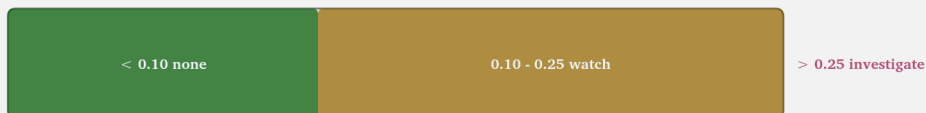
Figure 3.9. Rollout ladder: shadow (observes, serves nothing), then canary (small live fraction vs control), then full (staged 25/50/100%). Arm a tested one-command rollback before the canary starts.

How to read this diagram

1. Rung 1, shadow: the new version observes live traffic and serves nothing. Zero blast radius.
2. Rung 2, canary: a small live fraction runs against the control. Detection starts here.
3. Rung 3, full: staged 25, then 50, then 100 percent of traffic.
4. The rollback: armed and tested before the canary starts. Target: the previous version restorable in under 15 minutes.
5. What it is: the ordered path to production. Why it exists: each rung bounds the blast radius. What breaks: an untested rollback will not work when needed.

Figure 3.6. PSI: < 0.10 none, 0.10-0.25 watch, > 0.25 investigate

population stability tells you the world moved



PSI compares the live feature distribution to the training distribution. Above 0.25 the model is scoring a world it never met: investigate before the metrics do.

Run PSI on a schedule, not after an incident. Drift detected early is a retrain; detected late is an outage.

Figure 3.6. Population stability compares the live feature distribution to the training distribution. Above 0.25 the model is scoring a world it never met. Run PSI on a schedule, not after an incident: drift detected early is a retrain, detected late is an outage.

What to look at first: the three colored zones. The one takeaway: the world moves under the model; measure the movement.

3.6 Calendar and threshold numbers

These are the constants of the role. Memorize them as a set:

Number	Meaning
2-6 weeks	Security review duration; start it in week one
14-21 days	Target: working prototype on real customer data (time to value)
100-300	Golden-set cases to start (confidence vs labeling effort)
3-5	Customer experts for blind grading
2 weeks	Pilot shadow before any go-live decision
10-30 points	Typical demo-to-acceptance accuracy gap; never quote the demo number after the eval exists
1	Retries per model call, never a loop (retry storms take down systems)
48 hours	Post-mortem shared with the customer after an incident
15 minutes	Rollback target for the previous version
90% / 95% / 99.9%	Example quality / latency / availability SLOs (negotiate per engagement, then write them in the memo)

Next: Chapter 4 catalogs the classic traps: five failure shapes with their dodges.

\n

4. Classic traps

\n

Chapters 2 and 3 describe the engagement working. This chapter describes it failing. Five traps, each with the mechanism and the dodge. Every one of them has killed a real pilot.

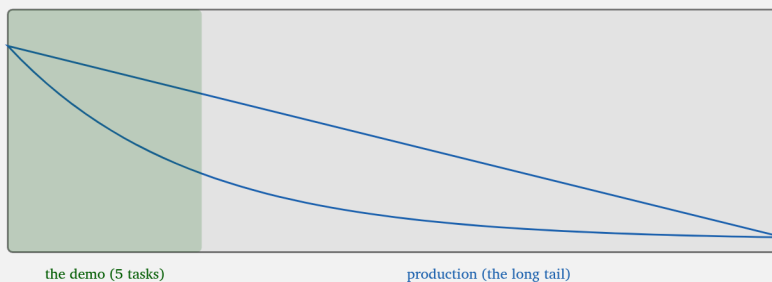
Trap 1: The demo-vs-production gap

Mechanism. The demo runs on clean sample data, one friendly user, admin permissions, one request at a time, errors on your screen. Production is dirty data, tired users at 6pm, partial permissions, spiky traffic, errors paging someone at 2am. The accuracy gap between demo cases and acceptance cases is typically **10-30 points**.

How to dodge. Promise less than the demo shows and write down exactly what the demo did not cover (the section 2.1 rule). Never quote the demo number after the acceptance eval exists. Run the 12-item hardening checklist as a gate on every release. Check: dirty-data pass, input limits, permission checks, secrets, audit logging, rate limits. Then: fall-backs, monitoring, rollback plan, green acceptance eval, runbook, handover training.

Figure 4.1. Trap 1: the demo-vs-production gap

the demo shows the head; production lives in the tail



The demo works because the five tasks were chosen. Production fails on task six through six hundred: novel inputs, hostile inputs, and Tuesday.

The fix: eval on a sampled slice of real traffic, not on the demo script.

Figure 4.1. The demo works because the team chose the five tasks. Production fails on task six through six hundred: novel inputs, hostile inputs, and Tuesday. The fix: evaluate on a sampled slice of real traffic, not on the demo script.

What to look at first: the green demo zone against the long blue tail. The one takeaway: the demo shows the head; production lives in the tail.

Trap 2: Scoping without kill criteria

Mechanism. The engagement starts with a vague ask and nobody writes down when to stop. Without kill criteria, every struggling engagement becomes a zombie: too much sunk cost to kill, too little progress to ship. The customer loses trust slowly, then all at once.

How to dodge. Write kill criteria before building. Give each one a number and a date. Get the sponsor to sign them. Example (on real data): "If the acceptance eval stays below 80 percent after 4 weeks, stop and reassess." Treat a triggered kill criterion as a success. The most trusted engineers kill fast and say why.

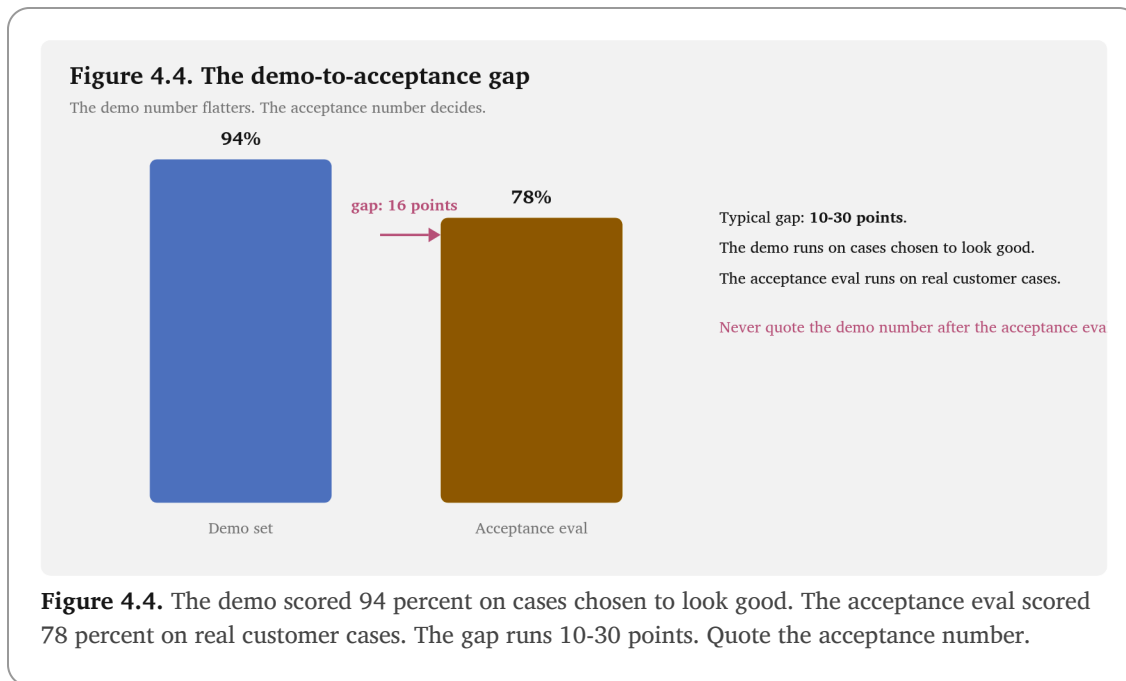
Trap 3: Evals that do not match the customer's definition of done

Mechanism. You report 94 percent on your eval set. The VP asks "94 percent of what?" and says "those are not our real cases." The eval measured the wrong thing. It used sample data instead of real cases. Its grader was unreadable to the customer.

It had no critical-failure rule. Its averages hid the catastrophe. Nothing ships because the measurement is not trusted.

How to dodge. Build the golden set from real customer cases with the customer's own experts (100-300 to start). Use deterministic graders first. Agree the pass threshold plus the no-critical-failure rule in advance.

Categorize every miss. Re-run on every change. Run the golden set as the section 2.4 contract: agreed before building, co-signed by the customer.



Trap 4: Silent cost blowups

Mechanism. The pilot is cheap: low volume, one model, nobody watching the bill. Then volume grows 10x, someone enables the large model for all traffic, retries multiply on every timeout, and the invoice arrives. Token cost was the only line anyone modeled; human review, rework, infra, and build amortization were invisible.

How to dodge. Price cost per successful outcome before the pilot, with all five parts. Set per-user and per-team quotas so the quarter-end flood degrades gracefully. Alert on cost per task, not just total spend. Recompute the breakeven volume whenever the model or the volume changes. Retry once, never in a loop.

Figure 4.2. Trap 4: silent cost blowups

the invoice is the sum of the costs nobody ledgers



the \$6 task: each step looks cheap in isolation

The fix from section 3.3: a token budget per task set from task value, a ledger enforced inside the loop, and a monthly cost table.

Figure 4.2. The invoice is the sum of the costs nobody ledgers: inference, retries, a critic on every step, the human review queue, and rework after failures. Each step looks cheap in isolation; the 25-step task costs \$6 and nobody notices until the invoice. The fix: a token budget per task from task value and a ledger inside the loop.

What to look at first: the five stacked cost bars, all larger than the inference bar alone.
The one takeaway: ledger every cost or the invoice ledgers them for you.

Trap 5: The permission check in the wrong place

Mechanism. The retrieval step runs first. The model reads everything. The permission filter applies afterward, "for display" only. An agent that surfaces a document the user may not see has caused a security incident, not shipped a feature. The demo ran as admin, so nobody noticed until the pilot's first hour.

How to dodge. Design the permission check before the retrieval step, never after (the section 2.2 rule). Filter by the requesting user's permissions before the model sees anything. Test with a low-privilege test user.

Next: Chapter 5 tests the material: 28 rapid-fire questions with worked answers.

\n

Figure 4.3. Trap 5: the permission check in the wrong place

a prompt cannot stop an action; a tool wrapper can

WRONG: in the prompt

"ask the user before refunding"; the model may comply,
forget, or be talked out of it

RIGHT: in the tool layer

the refund tool requires an approval token; no token,
no call, no debate

enforcement lives with the side effect, not with the suggestion

Review gates on risky actions need a 4-hour queue SLA, or the queue becomes the reason people bypass the gate.

Audit: list every tool with a side effect and confirm each one checks permission in code.

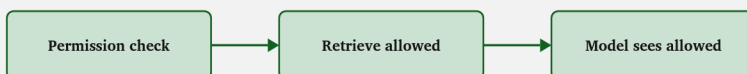
Figure 4.3. A prompt cannot stop an action; a tool wrapper can. "Ask the user before refunding" in the prompt can fail: the model may forget it or talk it away. The refund tool requiring an approval token enforces it in code: no token, no call, no debate. Review gates on risky actions need a 4-hour queue SLA, or the queue becomes the reason people bypass the gate.

Figure 4.5. The permission check goes BEFORE retrieval

Enforcement lives with the data flow, not with the suggestion.

WRONG: filter after retrieval

The model already saw restricted data. Incident, not a feature.

RIGHT: check before retrieval

Filter by the requesting user's permissions before the model sees anything.

Test with a low-privilege test user. The demo ran as admin.

Figure 4.5. Wrong: retrieve, let the model read everything, then filter for display. The model already saw restricted data. Right: check permissions first, retrieve only allowed documents, and let the model see only those.

How to read this diagram

1. Wrong flow, top: retrieval runs first. The model reads everything, including restricted documents. The filter applies afterward, for display only. The damage is done.
2. Right flow, bottom: the permission check runs first. Retrieval returns only allowed documents. The model never sees the rest.
3. What it is: the ordering rule for access control in RAG and agents. Why it exists: display filtering leaves data in model context, logs, and caches. What breaks: a prompt-level filter leaks through summaries, embeddings, or logs.

What to look at first: the red wrong box versus the green right box. The one takeaway: enforcement lives with the side effect, not with the suggestion.

5. Rapid-fire Q&A

\n

This chapter is the checkpoint: 28 questions across scoping, architecture, evals, and economics. Read each question, commit to an answer, then reveal. Every answer shows the working.

Twenty-eight checks. Read the question, commit to an answer, then reveal. Every answer shows the working.

Scoping

Q1. An engagement starts with "we want AI for our support team." What are the first three moves, in order, and why that order? A. Prototype, scope, discover: build fast to learn what the customer wants. B. Watch, ask, map: shadow users, run the question bank, draw the workflow. C. Ask, map, watch: questions first, then verify by shadowing.

Answer: B. Watching first makes your questions specific (you ask about the ticket queue you saw, not "tell me about support"). Mapping last forces you to commit to what you learned: each box marked automate, assist, or leave alone. Proposing a technical approach before the map exists is the most expensive mistake on customer work. A week of good discovery saves three months of building the wrong thing.

A, wrong: a prototype before discovery optimizes for the wrong task; you learn fast about a solution nobody asked for.

C, wrong: questions without observation stay generic; nobody asked the customer precisely, so your questions need the grounding that shadowing gives.

Q2. Which discovery question is the highest-value one on the bank, and why? A. "What does a perfect result look like?" because you need it for the golden set. B. "What did the last attempt at this fail on?" because it reveals the landmines. C. "If this works perfectly, what changes for the business?" because you need the ROI number.

Answer: B. Customers rarely volunteer failure history. Ask what the last attempt failed on, and ask what the vendor did wrong, not just what the technology did wrong. The answer is usually about trust, not accuracy, and it names the exact failure mode your kill criteria and evals must cover. (C matters for the economics chapter, and A feeds the golden set, but neither prevents you from stepping on the mine the last vendor found.)

A, wrong: valuable, but the golden example does not tell you what killed the last pilot.

C, wrong: the ROI number enables the cost model, but a wrong ROI assumption is survivable; repeating the last failure is not.

Q3. Which of these is a well-formed success criterion? A. "The system improves support efficiency by the end of the quarter." B. "By week 8, the system drafts responses for 60 percent of tier-1 tickets. Agents accept the draft unchanged at least 70 percent of the time, measured on live traffic." C. "The system reaches 95 percent accuracy on the demo set."

Answer: B. It has a number, a date, a data source, and it is measurable on the customer's real work. A cannot fail, which means it cannot succeed either. C measures the wrong distribution: the team chose demo cases to look good, and the demo-to-acceptance gap runs 10-30 points.

A, wrong: no number, no date, no measurement method. A wish, not a criterion.

C, wrong: the demo set is not the customer's definition of done; quote the acceptance number once the eval exists.

Q4. Which of these is a valid kill criterion? A. "We will reassess if the project is not going well." B. "If after 4 weeks on real data the acceptance eval stays below 80 percent, we stop and reassess." C. "The sponsor may cancel at any time with two weeks notice."

Answer: B. It names a metric, a threshold, a timebox, and a data source, so it can actually trigger. Kill criteria protect both sides. Write them before building and get the sponsor to sign them. A never triggers because "not going well" is arguable forever. C is a contract clause, not a decision rule: it says who may stop, not when the evidence says stop.

A, wrong: unfalsifiable; zombie engagements live on sentences like this.

C, wrong: describes authority, not evidence.

Enterprise architecture

Q5. Why is data classification the first question in pattern selection? A. Because the security team requires the paperwork first. B. Because residency and restriction answers can veto entire architectures before any other design work matters. C. Because it determines the model choice.

Answer: B. "This data cannot leave the country" kills the vendor-API pattern outright; "no internet path" forces on-premises. Everything downstream (where the model runs, what the latency looks like, what it costs) follows from classification. Asking it in week eight means redesigning. It does not pick the model (C); the same model can run in four patterns.

A, wrong: paperwork follows the decision; the decision is architectural, not bureaucratic.

C, wrong: classification constrains placement, not model selection.

Q6. A bank: customer data cannot leave its network, there is an internet path, and volume is steady and high. Which pattern, and why? A. Vendor API: fastest to value, and speed matters most. B. Private cloud VPC: data stays in-region, dedicated GPUs beat per-token pricing at steady high volume. C. On-premises: maximum control over the hardware.

Answer: B. The hard constraint (data stays in network) vetoes the vendor API before scoring; that is what vetoes are for. Between VPC and on-premises, steady high volume justifies dedicated GPU capacity in the customer's cloud account. Setup takes weeks, not the months of on-prem procurement. C is viable but slower to value with no compensating need (there is an internet path to their cloud).

A, wrong: vetoed by the hard constraint. No weight rescues a vetoed pattern.

C, wrong: acceptable only if the internet path did not exist or regulation demanded air-gapping.

Q7. The vendor API scores highest on your weighted table, but the customer's hard constraint is "data cannot leave the network." What do you recommend? A. The vendor API, because the math says it wins. B. Re-weight speed higher and re-run until the API wins honestly. C. Drop the vendor API: hard constraints are vetoes, and you score only the survivors.

Answer: C. A hard constraint eliminates options entirely; scoring ranks what survives. This mirrors how security reviews actually work: "data cannot leave the network" is not a preference to trade off. Weights are opinions you change live with the customer; vetoes are facts you do not negotiate with arithmetic.

A, wrong: the score never got to vote; the pattern was already eliminated.

B, wrong: tuning weights to rescue a vetoed pattern decorates a decision the security team will reverse in week six.

Q8. Where does the permission check go relative to retrieval, and why? A. After retrieval, filtering what gets displayed: simpler to do. B. Before retrieval: the model must never see what the user may not see. C. It does not matter as long as the UI hides restricted documents.

Answer: B. An agent that surfaces a document the user is not allowed to see has caused a security incident, not shipped a feature. Filtering after retrieval means the model already read restricted content (it can leak through summaries, embeddings, or logs). Test with a low-privilege test user; the demo ran as admin, which is why this breaks in the pilot's first hour.

A, wrong: display filtering leaves the data in model context, logs, and caches.

C, wrong: UI hiding is not access control.

Evals as contracts

Q9. Why start the golden set at 100-300 cases? A. It is a round number customers find reassuring. B. Below 100 the confidence intervals are too wide to argue about; above 300 the customer's experts will not finish labeling. C. Model judges need at least 100 examples to calibrate.

Answer: B. The lower bound is statistics (section 2.4: below 100 cases the confidence intervals are too wide to argue about). The upper bound is human: the customer's experts own the labels, and their time is the scarcest resource on the engagement. Real customer cases, not synthetic, at first.

A, wrong: reassurance is not a sizing method.

C, wrong: calibration is a grader concern, and deterministic graders come first anyway.

Q10. The system scores 95 percent on the golden set but fails one critical case: it leaked a salary number into a summary. Pass or fail? A. Pass: 95 percent clears the 90 percent threshold. B. Fail: the no-critical-failure rule fails the whole eval regardless of the average. C. Pass with a warning: note it in the taxonomy and move on.

Answer: B. Check the no-critical-failure rule as a separate boolean, never folded into the average, because customers care more about the worst case than the average. A 95 percent system that leaks one salary number is a failed system; customers remember the catastrophe, not the mean. The taxonomy (C) is for categorizing misses, not for waiving the rule.

A, wrong: the threshold was "90 percent AND no critical failures." Half the contract.

C, wrong: a warning does not un-leak the salary number.

Q11. The demo scored 94 percent; the acceptance eval scores 78 percent. Which number do you report to the sponsor? A. 94 percent: it shows the system's potential. B. 78 percent: the acceptance number is the only number once the eval exists. C. The average, 86 percent: it balances optimism and realism.

Answer: B. The demo ran on cases chosen to look good. The acceptance eval runs on cases the customer's experts chose to be representative, including the ugly ones. The gap is typically 10-30 points and it is where engagements are won or lost. Quoting the demo number after the acceptance eval exists destroys trust the first time reality disagrees.

A, wrong: potential is not a measurement; the sponsor will anchor on it and feel betrayed.

C, wrong: averaging two different distributions produces a number that measures nothing.

Q12. What is the purpose of the adversarial UAT (user acceptance testing) session, and where does it sit in the order? A. First: let skeptics try it early so they feel heard. B. Second, after blind grading: give skeptics 30 minutes to break it on purpose; every break becomes a golden case or a documented limit. C. Last, after the pilot shadow: stress-test before go-live.

Answer: B. Order: blind grading (kills bias both ways), adversarial session, pilot shadow. Skeptics who break the system in the room become advocates when it survives. Each break hardens the system at the exact place it was weak. Running it last (C) wastes the pilot's learning; running it first (A) lets bias ("it is AI, so it is wrong") substitute for evidence.

A, wrong: feelings without the blind-graded baseline first.

C, wrong: by then the breaks cost pilot trust instead of workshop time.

Prototype to production

Q13. Name the five ways demos die on real data. A. Dirty data, accidental adversarial users, permissions, traffic shape, unowned failures. B. Slow GPUs, bad prompts, small context, high temperature, wrong model. C. Scope creep, sponsor churn, budget cuts, vendor lock-in, team turnover.

Answer: A. The five are specific and repeatable:

- Scanned PDFs with coffee stains.
- Users pasting 50 pages into a 5-page box.
- The demo ran as admin.
- Monday-morning spikes and quarter-end floods.
- Errors with no runbook and no owner.

B lists model-tuning issues, which are real but not what kills demos. C lists organizational risks, handled in the communication chapter, not the hardening checklist.

Q14. Your fuzzer sends a 200,000-character paste. The system takes 6 seconds, then returns "Input too large." Clean or dirty failure? A. Clean: it returned the right message. B. Dirty: it hung for 6 seconds first; the probe demands a fast reject under 5 seconds with a clear message. C. Clean: 6 seconds is acceptable for an oversized input.

Answer: B. A clean failure is fast and clear: reject or truncate with a message in under 5 seconds (the number comes from the latency SLO). A 6-second hang on an adversarial input is the failure mode that pages someone at 2am when a user pastes the entire archive. The fuzzer exists to find this on a Tuesday afternoon instead.

A, wrong: the message was right but the timing violated the contract.

C, wrong: "acceptable" is not a number; the SLO is.

Q15. A user double-clicks submit and gets billed twice. Which property did the design miss? A. Low latency: the first response was too slow, so they clicked again. B. Idempotency: the system must detect the duplicate submission. C. Caching: the second request should have hit the cache.

Answer: B. The system must recognize the second identical submission as a duplicate (idempotency keys, dedup on the marker) instead of processing it as new work. Double-billing a customer because they double-clicked is the kind of bug that ends engagements. Latency (A) explains the click, not the double charge; caching (C) is a performance lever, not a correctness property.

Q16. What is the rollback requirement before the pilot? A. Rollback is nice to have; the pilot is small. B. Previous version restorable in under 15 minutes, with in-flight work preserved. C. Rollback within the 4-hour maintenance window.

Answer: B. Arm and test the rollback before the canary starts, not during the incident: an untested rollback will not work when you need it. Fifteen minutes bounds the blast radius; preserving in-flight work avoids turning a rollback into data loss. A treats the pilot as consequence-free, which it is not: pilot users are real users.

A, wrong: "small pilot" still pages someone when it breaks.

C, wrong: four hours of broken pilot is an incident, not maintenance.

Unit economics

Q17. Work it: model cost \$0.05/task, fixed shares \$0.30/task, success rate 80%, review 2 min, rework 10 min, labor \$75/hr. Cost per successful outcome? A. \$4.85 B. \$6.06 C. \$5.35

Answer: B. Labor = $0.80 \times (2/60) \times 75 + 0.20 \times (10/60) \times 75 = 2.00 + 2.50 = \4.50 . Per attempt = $0.05 + 0.30 + 4.50 = \$4.85$. Divide by the success rate: $4.85 / 0.80 = \$6.06$ per successful task.

A, wrong: that is cost per attempt; you skipped the division by the success rate, the step that converts attempts into outcomes.

C, wrong: check the arithmetic: rework labor is $0.20 \times 12.50 = 2.50$, not 1.80.

Q18. Model A costs \$2 per million tokens; Model B costs \$10 per million. Which is cheaper for the customer? A. Model A: 5x cheaper per token. B. Cannot tell from token price: compare cost per successful outcome, which includes success rate and human rework. C. Model B: expensive models are usually more reliable, so total cost is lower.

Answer: B. Tokens are not outcomes. The track's worked example shows a 6x token-price gap reversing. The pricey model at 92% success costs \$4.29 per successful task vs \$13.92 for the cheap model at 70%, because rework labor dominates. Always compare cost per successful outcome with success rates from your acceptance eval.

A, wrong: the token-price fallacy, the most common pricing mistake on engagements.

C, wrong: "usually" is not a measurement; run the model with measured rates.

Q19. Work it: fixed costs \$12,000/month, human baseline \$12.50/task, AI variable cost \$4.29 per successful task. Breakeven volume? A. 960 tasks/month B. About 1,460 tasks/month C. 2,796 tasks/month

Answer: B. $n = \text{fixed} / (\text{human} - \text{var}) = 12,000 / (12.50 - 4.29) = 12,000 / 8.21 \approx 1,461 \text{ tasks/month}$. Below that volume the fixed costs dominate and the human wins; above it the AI pulls ahead. Saying so early is how you keep trust with a small team.

A, wrong: that divides by the human cost alone, ignoring that the AI also has variable cost.

C, wrong: that adds the costs instead of taking the difference.

Q20. A vendor product claims 95% success at \$3 per task with zero build cost. How do you model the buy option? A. Take the 95% at face value: 0.95 goes straight into the comparison. B. Discount the claimed rate by your skepticism, validate it in the pilot shadow on customer data, then compare with zero build cost. C. Reject it: vendor claims are never trustworthy.

Answer: B. Buy is an option with zero build cost and the vendor's price as the model cost. Measure the success rate, do not quote it: vendor benchmarks run on their distribution, not the customer's. Validate in the pilot shadow, then run the same cost-per-outcome math. The usual honest answer is hybrid: buy the commodity model, build the evals, integration, and permission layer.

A, wrong: a success rate from the wrong distribution makes every downstream number fiction.

C, wrong: reflexive rejection leaves money on the table; measure instead.

Q21. In the five-part cost model, which line is usually the largest and the most forgotten? A. Model cost: tokens add up fast at scale. B. Human cost: review time, exception handling, and on-call. C. Infrastructure: GPUs are expensive.

Answer: B. An hour of reviewer time costs more than ten thousand model calls, and teams consistently forget to model it. Measure review minutes in the pilot shadow (round 3 of UAT); people are optimistic about review speed by 2-3x when estimating in a meeting. Model cost (A) is the most visible line, which is why it gets over-weighted; infra (C) matters but amortizes over volume.

SLOs and reliability

Q22. Work it: availability SLO 99.9%, and 5 of 1,000 requests go unanswered in the window. Burn rate and action? A. 0.5x: healthy, no action. B. 5.0x: pause feature work per the scoping-memo rule. C. 2.0x: watch closely.

Answer: B. $\text{Burn} = \text{observed} / \text{allowed} = (5/1,000) / 0.001 = 0.005 / 0.001 = 5.0x$. Above the 2x line, the rule fires: feature work pauses until the burn drops under 1x. The team agreed the rule in the scoping memo. The script just reads the dashboard, which is why the conversation is about the number instead of feelings.

A, wrong: inverted the fraction; 5 failures per thousand is 5x the 1-per-thousand budget.

C, wrong: 2.0x would be $(2/1,000)/0.001$.

Q23. Which latency statistic does the SLO use, and why? A. The mean: it summarizes all requests in one number. B. p95/p99: AI latency has a long tail, and the mean hides the requests the executives actually tried. C. p50: the median user experience is what matters.

Answer: B. Long inputs generate long outputs, so p50 looks fine while p95 is terrible. The average hides the exact requests the customer's executives tried in the demo, which are the ones they will complain about. SLO on the tail; also clarify whether the SLO names time-to-first-token or full generation (256 output tokens sets a physics floor near 5.4 s for full generation).

A, wrong: the mean is the statistic most easily gamed by a good median and most blind to the tail.

C, wrong: the median is honest but still blind to the tail where the pain lives.

Q24. The quality budget burns at 2.4x with three weeks left in the month. What does that imply? A. Nothing yet: the budget is not breached. B. It will breach before the month ends at this pace; freeze features now per the agreed rule. C. Recompute next week; one fast week might be noise.

Answer: B. Burn rate above 1.0 means breaching early; at 2.4x with three weeks left, the breach lands before the month ends. Alert on burn rate, not on breach. A team that waits for the breach waits too long. The freeze is not a negotiation; the team agreed it in the scoping memo, which is what gives the dashboard teeth.

A, wrong: "not breached yet" is how teams sleepwalk past the point of no return.

C, wrong: burn rate already accounts for pace; delaying the decision spends the remaining budget.

Q25. A model call times out. What is the correct fallback order? A. Retry in a loop until it succeeds, then page someone. B. Retry once, then a smaller/faster model, then a cached or template answer, then a clean human handoff with full context. C. Immediately hand off to a human; automation already failed.

Answer: B. Order the cascade by cost and reliability, and agree the trade-off in advance. Each step costs less and works more reliably than the last, usually at lower quality. Retry once, never in a loop: retry storms are how one slow model takes down the whole system. Critical tasks (sending a contract, a customer-facing message) skip the cascade sideways into mandatory human review. The handoff carries full context: what the team tried, what failed, the partial work so far.

A, wrong: unbounded retry is the retry-storm pattern.

C, wrong: skipping the cheaper fallbacks burns human time on every transient blip.

Figure 2.11. The weekly status word

Status word first, same format every week. Never bury it.



Regularity builds trust faster than good news.

Never let the customer discover a problem before you tell them.

Figure 2.11. Status word first, same format every week: GREEN, YELLOW, or RED. YELLOW with a mitigation plan is competence. RED buried in paragraph four is how trust dies.

How to read this diagram

1. **Read the word first: it sits at the top of the weekly update, in the same format every week.**
2. **GREEN: the work is on track. Ship the update and move on.**
3. **YELLOW: a risk exists, plus a mitigation plan. This is competence, not failure.**
4. **RED: the work hit a blocker or breached a bar. Report it early and first, from you.**
5. **What it is: the weekly trust contract. Why it exists: sponsors skim. What breaks: burying RED trains the sponsor to feel ambushed.**

Stakeholder communication

Q26. Where does the status word go in the weekly update, and what makes YELLOW a sign of competence rather than failure? A. At the end, after the evidence; YELLOW means the week went badly. B. First line, always honest; YELLOW with a mitigation plan and named owner. C. Only when the status changed; no news is good news.

Answer: B. Status word first, same format every week: GREEN / YELLOW / RED. YELLOW with a mitigation plan is competence; RED buried in paragraph four is how trust dies. Never let the customer discover a problem before you tell them. "You will hear this from me first" is a promise you make once and keep forever. Regularity builds trust faster than good news.

A, wrong: burying the status word trains the sponsor to skim past the bad news, then feel ambushed.

C, wrong: silence reads as either hiding or coasting; the cadence is the product.

Q27. A customer asks to go live next week; the eval is at 78% against a 90% bar.

What are the four parts of the "no"? A. Apologize, explain the technology, promise it will be ready soon, offer a discount. B. Direct answer, reason in their terms, what you can do instead, cost of the alternative. C. Say yes to protect the relationship, then work overtime to make it true.

Answer: B. "We cannot go live next week responsibly. At 78%, roughly 1 in 5 reviews would need full rework, which adds load to your team instead of removing it. Instead: two more weeks of pilot shadow while we fix the two failing categories, reaching the 90% bar we agreed on. Cost: two weeks of timeline, no additional cost." Every no comes with an instead and a price tag. C destroys the relationship later to protect it now; A has no instead.

A, wrong: apology plus jargon plus a vague promise is not a plan.

C, wrong: saying yes to an infeasible ask is the most expensive sentence on an engagement.

Q28. Which asks must you always refuse, no matter how the customer frames them?

A. 100% accuracy, skipping the security review, go-live without a passing eval, using data the customer has no rights to. B. Fixed-price contracts, on-premises deployment, weekly status updates, kill criteria. C. Anything the sponsor's boss requested; the sponsor is your only customer.

Answer: A. For the first three, offer the alternative in the same breath. Name the measured rate plus the failure taxonomy. Start the review today in parallel. Offer the pilot shadow. The fourth (unlicensed data) is just no.

A, right: 100% accuracy is unachievable. A skipped security review becomes a week-eight blocker. Go-live without a passing eval ships the catastrophe. Unlicensed data is a legal incident.

B, wrong: every item listed is legitimate; a refusal here refuses the job.

C, wrong: the sponsor's boss, security, legal, and IT are all stakeholders; they get precision and lead time, not refusal.

Next: Chapter 6 compresses everything into a one-page cheat sheet.

\n

6. One-page cheat sheet: 30 numbers and rules

\n

Everything above, compressed to one page. Skim it before a customer meeting, keep it open during scoping.

#	Number / rule	Why it matters
1	Discovery is watch, ask, map. Never propose tech before the map.	A week of discovery saves three months of building the wrong thing.
2	"What did the last attempt fail on?" is the highest-value question.	Names the landmine the last vendor found; usually about trust, not accuracy.
3	Success criteria need numbers plus dates, measurable on customer data.	A criterion that cannot fail cannot succeed.
4	Write kill criteria before building. Sign them. Make them specific enough to trigger.	Killing in week 2 costs a week; in month 4 it costs the relationship.
5	Data classification picks the deployment pattern.	Residency answers can veto entire architectures.
6	Hard constraints are vetoes, not weights. Score only survivors.	No weight rescues a pattern the security review will kill.
7	Security review takes 2-6 weeks; start it in week one.	Bring the data-flow diagram, data inventory, and permission model.
8	Permission check before retrieval, never after.	The model must never see what the user may not see.
9	Golden set: 100-300 real customer cases, labeled by customer experts.	Below 100 the statistics are too weak; above 300 nobody finishes.
10	Graders deterministic first; model judge only if impossible, calibrated first.	A grader the customer cannot read is a grader they will not trust.
11	No-critical-failure rule: one catastrophic miss fails the eval.	Customers remember the catastrophe, not the average.
12	UAT: blind grading, then adversarial session, then 2-week pilot shadow.	Skeptics who break it in the room become advocates when it survives.
13	Demo-to-acceptance gap is typically 10-30 points.	Quote the acceptance number once the eval exists. Never the demo number.
14	Cost per successful outcome = per-attempt cost / success rate (\$/successful task).	Five parts: model, infra share, build share, maintenance share, human labor.

15	Success rates come from your acceptance eval, never vendor benchmarks.	Wrong distribution in, fiction out.
16	Breakeven volume = $\text{fixed_monthly} / (\text{human_per_success} - \text{var_per_success})$.	Below it the human wins; say so early.
17	Token price is not cost. Compare cost per successful outcome.	A 6x token gap reversed to \$4.29 vs \$13.92 in the worked example.
18	Human labor is usually the largest cost line.	Measure review minutes in the pilot shadow; estimates run 2-3x optimistic.
19	SLO latency on p95/p99, never the mean.	The tail is where the executives' demo requests live.
20	Error budget = $1 - \text{SLO}$. Burn rate = observed / allowed.	1.0 is on pace; above 1.0 breaches early.
21	Burn above 2x pauses feature work until under 1x.	The rule lives in the scoping memo; the dashboard just reads it.
22	99.9% availability = 43.2 minutes downtime per 30 days.	$0.001 \times 30 \times 24 \times 60$. Memorize it.
23	Fallback cascade: retry once, smaller model, cache, human handoff.	Critical tasks skip to human review. Retry in a loop never.
24	PSI: below 0.10 fine, 0.10-0.25 watch, above 0.25 investigate.	Drift is the quiet killer of quality SLOs.
25	Canary: 2,000-5,000 samples to tell 1% from 2%.	Size trades detection speed against blast radius.
26	Rollout: shadow, then canary, then staged full. Rollback under 15 min, tested first.	An untested rollback will not work during the incident.
27	Working prototype on real data by day 14-21.	Time to value buys the trust the hardening months spend.
28	Weekly update: status word first, honest, bad news from you first.	Regularity builds trust faster than good news.

29	Every no: direct answer, reason in their terms, instead, cost of the alternative.	"No, but here is what yes looks like" is the whole skill.
30	Post-mortem within 48 hours; every incident becomes a fuzzer probe and a golden case.	Customers forgive incidents. They do not forgive silence.

Last verified: September 2026. The following come from Role Track 7 (Forward Deployed Engineer / AI Consultant):

- Decision rules.
- Scoping method.
- Deployment patterns.
- Eval-as-contract anatomy.
- Hardening checklist.
- Unit-economics model.
- SLO framework.
- Fallback cascade.
- Communication templates.

Serving cost model, canary math, PSI thresholds, and cost levers reference Volumes 10 and 13. The author executed all Python snippets on this machine and verified their printed outputs.

UNVERIFIED: dollar figures in worked examples are illustrative, not measured (reviewer labor \$75/hr, human baseline \$12.50/task, infra/build figures). Vendor token prices shift frequently. Security-review durations and pilot timelines vary by customer.

Where to go next. This pack is done. The crash sequence continues with [Crash: AI Engineer](#), the applied-LLM-systems side of the same work: chunking, retrieval, evals, and cost math for production LLM systems. The unabridged version of everything here lives in [Track 07: Forward Deployed Engineer](#).

\n

Watch and go deeper

Forward Deployed Engineer: Hype, Reality and a Realistic Roadmap (Aishwarya Reganti)

18 minFDE role

Why this one: it is built on 1000+ real FDE job listings. It traces the role from Palantir's "Dev vs Delta" origin through four core responsibilities. Then it busts four myths with data, and the three-layer roadmap at the end is a study plan in disguise. **Watch for:** the myth-busting section. It answers the "is FDE just sales engineering?" question the traps in section 3 warn about. **Skip if:** you can already name the four responsibilities and the Palantir origin story from memory.

Transcript read: the full transcript is also published in her GitHub repo. It covers the listings analysis, the origin story, responsibilities, myths, and the layered roadmap.

What Is a Forward Deployed Engineer? (Founding Rippling FDE)

38 minFDE practice

Why this one: a practitioner who did the job at Palantir and as a founding FDE at Rippling. He draws the line between FDE work and sales engineering, and he answers the hype question honestly: where the role is real and where it is theater. **Watch for:** his stories about the first 90 days on site. They show what "deployment is the job" actually feels like. **Skip if:** the practitioner view adds nothing once you have the role mechanics down.

Transcript read: the conversation covers the Palantir background, the FDE vs sales-engineering distinction, and a direct take on the hype question.

Go deeper

- [FDE job board and skill map](#): the board behind the video's listings analysis; use the skill map to find your gaps against real postings.
- [Implementing SLOs](#) (Google SRE Workbook): the full SLO method behind section 3.2's error-budget math.
- [What is a Forward Deployed Engineer?](#) (Exponent): a practitioner-written role guide that pairs well with the engagement loop in section 1.